

Fiche de révision - Python pour le bac de mathématiques

1. À maîtriser pour le bac

- Python apparaît surtout dans des questions sur les suites et éventuellement les exercices sur les fonctions.
- À savoir lire : affectation, test, boucle `for`, boucle `while`, fonction, `return`.
- Savoir compléter une ligne de code ou interpréter le résultat donné par un algorithme.

2. Modèles à connaître

Calculer un terme u_n

```
def suite(n):          # fonction qui calcule le terme u_n
    u = 3              # on initialise avec le premier terme u_0
    for i in range(n): # on repete n fois la relation de recurrence
        u = 4/(5-u)    # on remplace u par le terme suivant
    return u           # on renvoie la valeur de u_n
```

Trouver un seuil

```
def seuil():           # fonction de recherche d'un seuil
    n = 0              # rang courant
    u = 1              # valeur initiale u_0
    while u <= 14:     # on continue tant que le seuil n'est pas depasse
        u = -0.02*u + 1.3*u # calcul du terme suivant
        n = n + 1      # on avance d'un rang
    return n           # premier rang pour lequel u > 14
```

Obtenir un rang et une valeur

```
def algo(p):           # fonction qui depend de la precision p
    u = 2              # valeur initiale
    n = 0              # nombre d'iterations effectuees
    while u - 1 > p:    # on boucle tant que l'ecart a 1 est trop grand
        u = (2*u + 1)/(u + 2) # calcul du terme suivant
        n = n + 1      # une iteration de plus
    return (n, u)      # on renvoie le rang et la valeur obtenue
```

Dichotomie

```
def racine(a, b):      # fonction qui travaille sur l'intervalle [a, b]
    while abs(b-a) >= 0.001: # on continue tant que l'intervalle est trop large
        m = (a+b)/2      # milieu de l'intervalle
        if f(m) < 0:     # test du signe de f au milieu
            a = m         # on garde la moitie droite
        else:            # sinon, la racine est dans l'autre moitie
            b = m         # on garde la moitie gauche
    return m             # valeur approchee de la racine
```

3. Erreurs classiques à éviter

- Mettre la condition du `while` dans le mauvais sens.
- Oublier une mise à jour : `u = ...` ou `n = n + 1`.
- Placer `return` dans la boucle alors qu'il doit souvent être après la boucle.
- Confondre le rang renvoyé et la valeur du terme.

4. Exercices type bac

Exercice 1 - Lecture d'une boucle for

On considère la suite (u_n) définie par $u_0 = 4$ et $u_{n+1} = 0,8u_n + 1,2$.

```
def terme(n):           # fonction qui calcule un terme de la suite
    u = 4                # valeur initiale u_0
    for i in range(n):   # on applique n fois la recurrence
        u = 0.8*u + 1.2  # calcul du terme suivant
    return u             # valeur renvoyee : u_n
```

Que renvoie `terme(0)` ? Que calcule la fonction `terme` ?

Exercice 2 - Compléter un script

Pour $u_0 = 1$ et $u_{n+1} = 1,5u_n + 2$, compléter la fonction qui renvoie u_n .

```
def suite(n):           # fonction qui doit renvoyer u_n
    u = ...              # valeur initiale u_0
    for i in range(...): # nombre de mises a jour a effectuer
        u = ...          # relation de recurrence
    return ...           # valeur a renvoyer
```

Exercice 3 - Interpréter return

```
def algo():             # fonction sans parametre
    n = 0                # rang courant
    u = 3                # valeur initiale
    while u < 20:         # on continue tant que le seuil 20 n'est pas atteint
        u = 1.1*u + 2    # calcul du terme suivant
        n = n + 1        # mise a jour du rang
    return (n, u)         # rang obtenu et valeur correspondante
```

Interpréter précisément les deux nombres renvoyés.

Exercice 4 - Plus petit rang

Pour $u_0 = 5$ et $u_{n+1} = 0,9u_n + 3$, compléter le script qui renvoie le plus petit entier n tel que $u_n > 20$.

```
def seuil():            # fonction qui cherche un seuil
    n = 0                # rang initial
    u = 5                # valeur initiale u_0
    while ..... :       # condition : le seuil n'est pas encore depasse
        .....           # calcul du terme suivant
        .....           # mise a jour du rang
    return n             # premier rang qui convient
```

Exercice 5 - Condition du while

On cherche le plus petit entier n tel que $u_n < 0,1$, avec $u_0 = 2$ et $u_{n+1} = 0,4u_n$. Quelle condition convient ?

- A. while u < 0.1:
- B. while u <= 0.1:
- C. while u >= 0.1:
- D. while n >= 0.1:

Exercice 6 - Valeur approchée d'une limite

Pour $u_0 = 5$, $u_{n+1} = \sqrt{u_n + 1}$ et $\ell = \frac{1 + \sqrt{5}}{2}$:

```
from math import sqrt          # pour utiliser la racine carree

def seuil(p):                  # fonction qui depend de la precision p
    u = 5                      # valeur initiale u_0
    i = 0                      # compteur d'iterations
    l = (1 + sqrt(5))/2        # valeur de la limite
    while abs(u-l) >= 10**(-p): # on continue tant que l'ecart est trop grand
        u = sqrt(u+1)          # calcul du terme suivant
        i = i + 1              # une iteration de plus
    return i                   # nombre d'iterations necessaires
```

Interpréter seuil(3).

Exercice 7 - Somme d'une suite

Pour $u_0 = 8$ et $u_{n+1} = u_n - \ln\left(\frac{u_n}{4}\right)$:

```
from math import log          # pour utiliser le logarithme neperien

def mystere(k):               # fonction qui depend du nombre k de termes
    u = 8                     # valeur initiale u_0
    S = 0                     # somme initialement nulle
    for i in range(k):        # on repete k fois
        S = S + u             # on ajoute le terme courant a la somme
        u = u - log(u/4)      # on calcule le terme suivant
    return S                  # somme des termes ajoutees
```

Que représente mystere(10) ?

Exercice 8 - Modifier un script

Reprendre l'exercice précédent. Modifier la fonction pour qu'elle renvoie la moyenne des k premiers termes au lieu de leur somme.

Exercice 9 - Encadrement par dichotomie

f est continue et strictement croissante sur $[1; 3]$; l'équation $f(x) = 0$ admet une unique solution α dans cet intervalle. Compléter cet algorithme de dichotomie qui doit renvoyer une valeur de $\alpha = 0.001$ près :

```
def racine(a, b):          # recherche d'une racine dans [a, b]
    while abs(b-a) >= 0.001: # precision souhaitee : largeur inferieure a 0.001
        m = ...             # milieu de l'intervalle
        if f(m) < 0:        # test du signe au milieu
            ...             # on remplace une borne
        else:               # autre cas possible
            ...             # on remplace l'autre borne
    return m               # valeur approchee de la solution
```

Exercice 10 - Encadrer une solution

Soit $f(x) = 4 \ln(1+x) - \frac{x^2}{25}$ sur $[2; 6,5]$.

```
from math import log      # pour utiliser le logarithme neperien

def bornes(n):            # fonction qui depend du nombre de decimales n
    p = 1/10**n           # pas de balayage : 10^(-n)
    x = 6                 # valeur de depart
    while f(x)-x > 0:      # on avance tant que f(x) est au-dessus de x
        x = x + p         # on augmente x d'un pas
    return (x-p, x)       # encadrement obtenu
```

Interpréter les deux nombres renvoyés par `bornes(2)`.

Exercice 12 - Compléter une récurrence

Pour $u_0 = 0$ et $u_{n+1} = 5u_n - 8n + 6$, compléter la fonction qui renvoie u_n .

```
def suite_u(n):           # fonction qui doit renvoyer u_n
    u = ...               # valeur initiale u_0
    for i in range(...):  # indices utilisés dans la recurrence
        u = ...           # formule donnant le terme suivant
    return u              # valeur de u_n
```

Exercice 13 - Script et contexte

On considère la suite définie par $u_1 = 3$ et $u_{n+1} = 1,1u_n + 1,3$.

```
def seuil(p):             # fonction qui cherche quand u depasse p
    n = 1                 # rang initial : on part de u_1
    u = 3                 # valeur initiale u_1
    while u <= p:         # on continue tant que le seuil n'est pas depasse
        n = n + 1         # on passe au rang suivant
        u = 1.1*u + 1.3   # calcul du terme suivant
    return n              # premier rang tel que u_n > p
```

Interpréter `seuil(8.5)`.

Exercice 14 - Somme

Que renvoie ce script ?

```
def suite(k):             # fonction censee calculer une somme
    S = 0                 # somme initiale
    for i in range(k):    # boucle avec l'indice i
        S = S + (3/4)**k  # on ajoute toujours la meme puissance
    return S              # somme obtenue par le script
```

5. Les listes

Les listes servent à mémoriser plusieurs valeurs au lieu de conserver seulement la dernière.

Syntaxes utiles. `L = []` crée une liste vide ; `L.append(x)` ajoute une valeur ; `L[0]` donne le premier élément ; `len(L)` donne la longueur.

Construire la liste des premiers termes

Pour une suite définie par $u_0 = 5$ et $u_{n+1} = 2 + \ln(u_n^2 - 3)$:

```
from math import log          # pour utiliser le logarithme neperien

def premiers_termes(k):       # fonction qui renvoie une liste de k termes
    L = []                   # liste vide au depart
    u = 5                    # valeur initiale u_0
    for i in range(k):       # on repete k fois
        L.append(u)          # on ajoute le terme courant dans la liste
        u = 2 + log(u*u - 3) # on calcule le terme suivant
    return L                  # on renvoie la liste complete
```

À retenir. Dans ce type de script, u calcule le terme suivant et L mémorise les termes déjà obtenus. Les indices de liste commencent à 0.

Tester numériquement une propriété

```
def semble_croissante(L):    # fonction qui teste une liste de valeurs
    for i in range(len(L)-1): # on parcourt les indices utiles
        if L[i+1] < L[i]:    # on compare deux termes consecutifs
            return False     # une baisse a ete trouvee
    return True               # aucune baisse n'a ete reperee
```

Mini-exercice

```
def termes(k):              # fonction qui renvoie une liste de termes
    L = []                   # liste vide au depart
    u = 3                    # valeur initiale u_0
    for i in range(k):       # on veut stocker k termes
        L.append(u)          # on memorise le terme courant
        u = 0.5*u + 4        # on calcule le terme suivant
    return L                  # liste des termes stockes
```

Que renvoie termes(4) ? Que représente la liste renvoyée ?